LEARN BEING FORWARD

Programming — C Programming

Arrays, strings & structures

COMPLETE LEARNING GUIDE

What You'll Learn

Arrays, strings, and structures are how C organizes multiple related values into a single unit, and all three reveal more of C's low-level memory model than equivalent features in higher-level languages: a C string is nothing more than a char array with a convention (a trailing null byte) rather than a distinct built-in type, and a struct's layout in memory can include invisible padding bytes the compiler inserts for alignment. Because C provides no bounds checking on arrays and no automatic memory management for strings, nearly every serious security vulnerability in C's history traces back to misused arrays or strings — buffer overflows from strcpy, off-by-one errors in loop bounds, missing null terminators. This sub-module covers array memory layout, multidimensional arrays, C string mechanics and the standard string-handling functions, the structure type, and struct memory layout.

1. Array Declaration and Memory Layout

A C array is a fixed-size, contiguous block of memory holding elements of the same type, with no metadata stored alongside it recording its own length — the programmer is entirely responsible for tracking how many elements an array holds, usually via a separate size variable or a #define'd constant. Because arrays are contiguous, element access via arr[i] is a single pointer-arithmetic calculation (constant time), and the array's name in most expressions decays to a pointer to its first element, which is why sizeof(arr) gives the TOTAL array size in bytes only when arr is still the actual array (not a function parameter it has decayed into).

- int arr[10]; sizeof(arr) is 40 (10 ints × 4 bytes) in the scope where arr is the real array — but decays to a pointer size once passed to a function.
- C performs NO bounds checking: arr[10] on a 10-element array (valid indices 0-9) reads/writes out-of-bounds memory silently, which is undefined behavior.

```c
#include <stdio.h>

int main(void) {
    int arr[5] = {10, 20, 30, 40, 50};
    printf("sizeof(arr) = %zu bytes\n", sizeof(arr));        // 20 (5 * 4)
    printf("number of elements = %zu\n", sizeof(arr) / sizeof(arr[0])); // 5
    return 0;
}
```

2. Multidimensional Arrays

A 2D array (int grid[3][4];) is still, under the hood, one single contiguous block of memory arranged in row-major order — all of row 0's elements, then all of row 1's, and so on — not an array of separate pointers to separate rows (that pattern, an array of pointers, is a different and more flexible structure often used when row lengths can vary). Accessing grid[i][j] is computed as *(grid + i * NUM_COLS + j) internally, which is why the number of columns must be known (or fixed) at compile time for a true 2D array, and why nested loops iterating row-then-column match the underlying memory layout for the best cache performance.

```c
#include <stdio.h>

int main(void) {
    int grid[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", grid[i][j]);
        }
    }
    printf("\n"); // 1 2 3 4 5 6
    return 0;
}
```

3. C Strings: char Arrays and the Null Terminator

C has no dedicated string type — a C string is simply a char array whose contents end with a null byte ('\0', value 0), and every standard string function relies on that null byte to know where the string ends. A string literal like "hello" automatically gets a null terminator appended by the compiler, making it 6 bytes total even though it displays 5 visible characters — forgetting to budget space for that extra byte when sizing a char buffer is one of the most common C beginner mistakes. strlen() counts characters up to (but not including) the null terminator, and does NOT include it in the returned count.

- char name[5] = "hello"; is a compile error/warning — "hello" needs 6 bytes (5 letters + null terminator), not 5.
- strlen("hello") returns 5, but the actual memory used is 6 bytes because of the trailing '\0'.

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char name[6] = "hello"; // exactly enough: 5 chars + '\0'
    printf("strlen = %zu\n", strlen(name)); // 5
    printf("sizeof = %zu\n", sizeof(name)); // 6
    return 0;
}
```

4. String Manipulation and the strcpy Buffer-Overflow Trap

<string.h> provides strcpy, strcat, strcmp, and strlen as the core string-handling toolkit, but strcpy and strcat perform NO bounds checking on the destination buffer — if the source string is longer than the destination buffer, they happily write past its end, corrupting adjacent memory (a classic, historically exploited buffer overflow). The bounded variants strncpy and strncat accept a maximum length argument and are safer, though strncpy has its own gotcha: it does not guarantee null-termination if the source is exactly as long as (or longer than) the limit, so the destination buffer should usually be explicitly null-terminated afterward as a safety net. Comparing strings must always use strcmp (which returns 0 for equality), never == (which compares pointer addresses, not string contents).

- if (str1 == str2) compares whether the two POINTERS are identical, not whether the strings' contents match — always use strcmp(str1, str2) == 0 instead.
- strncpy(dest, src, size) truncates safely but may leave dest without a null terminator if src is size characters or longer — add dest[size-1] = '\0'; defensively.

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char buffer[10];
    strncpy(buffer, "Hello", sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0'; // defensive null-termination

    if (strcmp(buffer, "Hello") == 0) {
        printf("Strings match\n");
    }
    return 0;
}
```

5. Structures: Defining and Using struct

A struct groups several related variables (possibly of different types) under one name, giving each member field its own name accessed with the dot operator (instance.field) or, through a pointer to the struct, the arrow operator (ptr->field, equivalent to (*ptr).field but far more readable). Structures are the primary way C models a single real-world 'thing' with multiple attributes — a point with x and y, a student with a name and a grade — and are passed to functions by value by default, meaning the entire structure is copied unless a pointer to it is passed instead (important for both performance on large structs and for functions that need to modify the caller's struct).

```c
#include <stdio.h>

struct Point {
    int x;
    int y;
};

void printPoint(const struct Point *p) { // pointer avoids copying the whole struct
    printf("(%d, %d)\n", p->x, p->y); // arrow operator through a pointer
}

int main(void) {
    struct Point origin = {0, 0};
    struct Point p1 = {3, 4};

    printf("%d\n", p1.x); // dot operator on a direct struct value
    printPoint(&p1);
    printPoint(&origin);
    return 0;
}
```

6. Nested Structures and Arrays of Structures

Structures can contain other structures as members (a Rectangle struct might hold two Point members for its corners), and arrays of structures are a common way to model a collection of records (a roster of Student structs). Accessing a nested field chains the dot operator (rect.topLeft.x), and iterating an array of structs with a for loop is the standard pattern for processing a list of records — each element is a full, independent copy of the struct, so modifying arr[i] never affects any other element.

```c
#include <stdio.h>
#include <string.h>

struct Student {
    char name[30];
    int grade;
};

int main(void) {
    struct Student class[3] = {
        {"Asha", 88},
        {"Ravi", 92},
        {"Meera", 79}
    };

    for (int i = 0; i < 3; i++) {
        printf("%s: %d\n", class[i].name, class[i].grade);
    }
    return 0;
}
```

7. typedef and Structure Padding

typedef creates an alias for a type, most commonly used with structs to avoid repeating the struct keyword everywhere (typedef struct Point Point; lets you write Point p; instead of struct Point p;). Separately, the compiler often inserts invisible padding bytes between struct members to satisfy each member's alignment requirement (a 4-byte int typically must start at an address divisible by 4) — this means sizeof(struct) can be larger than the sum of its members' individual sizes, and reordering members (largest-to-smallest is a common heuristic) can sometimes reduce the total padding and shrink the struct.

- sizeof(struct { char c; int i; }) is often 8, not 5 — 3 padding bytes are inserted after c so that i starts on a 4-byte boundary.
- typedef struct { int x, y; } Point; combines the definition and the alias into one statement, a very common idiom.

```c
#include <stdio.h>

typedef struct {
    int x;
    int y;
} Point; // Point is now usable directly, without writing "struct"

int main(void) {
    Point p = {5, 10};
    printf("(%d, %d)\n", p.x, p.y);
    return 0;
}
```

Common Pitfalls

- Sizing a char buffer for a string's visible length without the extra byte needed for the null terminator.
- Using strcpy/strcat on buffers of unknown or untrusted source length instead of the bounds-checked strncpy/strncat.
- Comparing C strings with == instead of strcmp(), which compares pointer addresses rather than string contents.
- Assuming sizeof(arr) still gives the full array size after the array has decayed into a pointer function parameter.
- Forgetting that a 2D array is one contiguous block in row-major order, not an array of independent row pointers.
- Assuming a struct's sizeof equals the exact sum of its members' sizes, ignoring compiler-inserted alignment padding.
- Passing a large struct by value to a function repeatedly instead of passing a pointer, incurring unnecessary copy overhead.

Key Takeaways

- A C string is a char array ending in a '\0' byte — always budget one extra byte of buffer space for it.
- strcpy/strcat perform no bounds checking; prefer strncpy/strncat and defensively null-terminate the result.
- String content comparison requires strcmp(); == on strings compares addresses, not contents.
- A 2D array is one contiguous row-major memory block, not an array of separate row pointers.
- struct members are accessed with . on a direct value and -> through a pointer (ptr->field is (*ptr).field).
- Compiler-inserted padding means sizeof(struct) can exceed the sum of its members' individual sizes.

Learn More

cppreference — Array declaration
https://en.cppreference.com/w/c/language/array

GeeksforGeeks — Structures in C
https://www.geeksforgeeks.org/c/structures-in-c/